CSCI467

Data Mining



Course Instructors:
Dr. Ibrahim Zaghloul

Lecture 4

Frequent Itemsets , Association Rules

outlines

- Frequent Itemsets
- Mining Frequent Itemsets task
- The Apriori Principle and algorithm
- ASSOCIATION RULES

This is how it all started…

- Rakesh Agrawal, Tomasz Imielinski, Arun N. Swami: Mining Association Rules between Sets of Items in Large Databases. SIGMOD Conference 1993: 207-216
- Rakesh Agrawal, Ramakrishnan Srikant: Fast Algorithms for Mining Association Rules in Large Databases. VLDB 1994: 487-499

- These two papers are credited with the birth of Data Mining
- For a long time people were fascinated with Association Rules and Frequent Itemsets
  - Some people (in industry and academia) still are.

Market-Basket Data

- A large set of items, e.g., things sold in a supermarket.
- A large set of baskets, each of which is a small set of the items, e.g., the things one customer buys on one day.
- Really, a general many-to-many mapping (association) between two kinds of things, where the one (the baskets) is a set of the other (the items)
  - But we ask about connections among “items,” not “baskets.”
- The technology focuses on common events, not rare events (“long tail”).

Applications – (1)

- Items = products; baskets = sets of products someone bought in one trip to the store.

- Example application: given that many people buy Juice and diapers together:
  - Run a sale on diapers; raise price of Juice.
- Only useful if many buy diapers & Juice.

Applications – (2)

- Baskets = Web pages; items = words.

- Example application: Unusual words appearing together in a large number of documents, e.g., “Brad” and “Angelina,” may indicate an interesting relationship.

Applications – (3)

- Baskets = sentences; items = documents containing those sentences.

- Example application: Items that appear together too often could represent plagiarism.
- Notice items do not have to be “in” baskets.

Definition: Frequent Itemset

- **Itemset**
  - A collection of one or more items
    - Example: {Milk, Bread, Diaper}
  - k-itemset
    - An itemset that contains k items
- **Support (σ)**
  - **Count**: Frequency of occurrence of an itemset
  - E.g. σ({Milk, Bread,Diaper}) = 2
  - **Fraction**: Fraction of transactions that contain an itemset
  - E.g. s({Milk, Bread, Diaper}) = 40%
- **Frequent Itemset**
  - An itemset whose support is greater than or equal to a *minsup* threshold

TID	*Items*
1	Bread, Milk
2	Bread, Diaper, Juice, Eggs
3	Milk, Diaper, Juice, Coke
4	Bread, Milk, Diaper, Juice
5	Bread, Milk, Diaper, Coke

$s(I) \geq \text{minsup}$

Frequent Itemsets

- Given a set of transactions, find combinations of items (itemsets) that occur frequently

Support $s(I)$: number of transactions that contain itemset I

Market-Basket transactions

Items: {Bread, Milk, Diaper, Juice, Eggs, Coke}

TID	*Items*
1	Bread, Milk
2	Bread, Diaper, Juice, Eggs
3	Milk, Diaper, Juice, Coke
4	Bread, Milk, Diaper, Juice
5	Bread, Milk, Diaper, Coke

Examples of frequent itemsets $s(I) \geq 3$

{Bread}: 4
{Milk} : 4
{Diaper} : 4
{Juice}: 3
{Diaper, Juice} : 3
{Milk, Bread} : 3

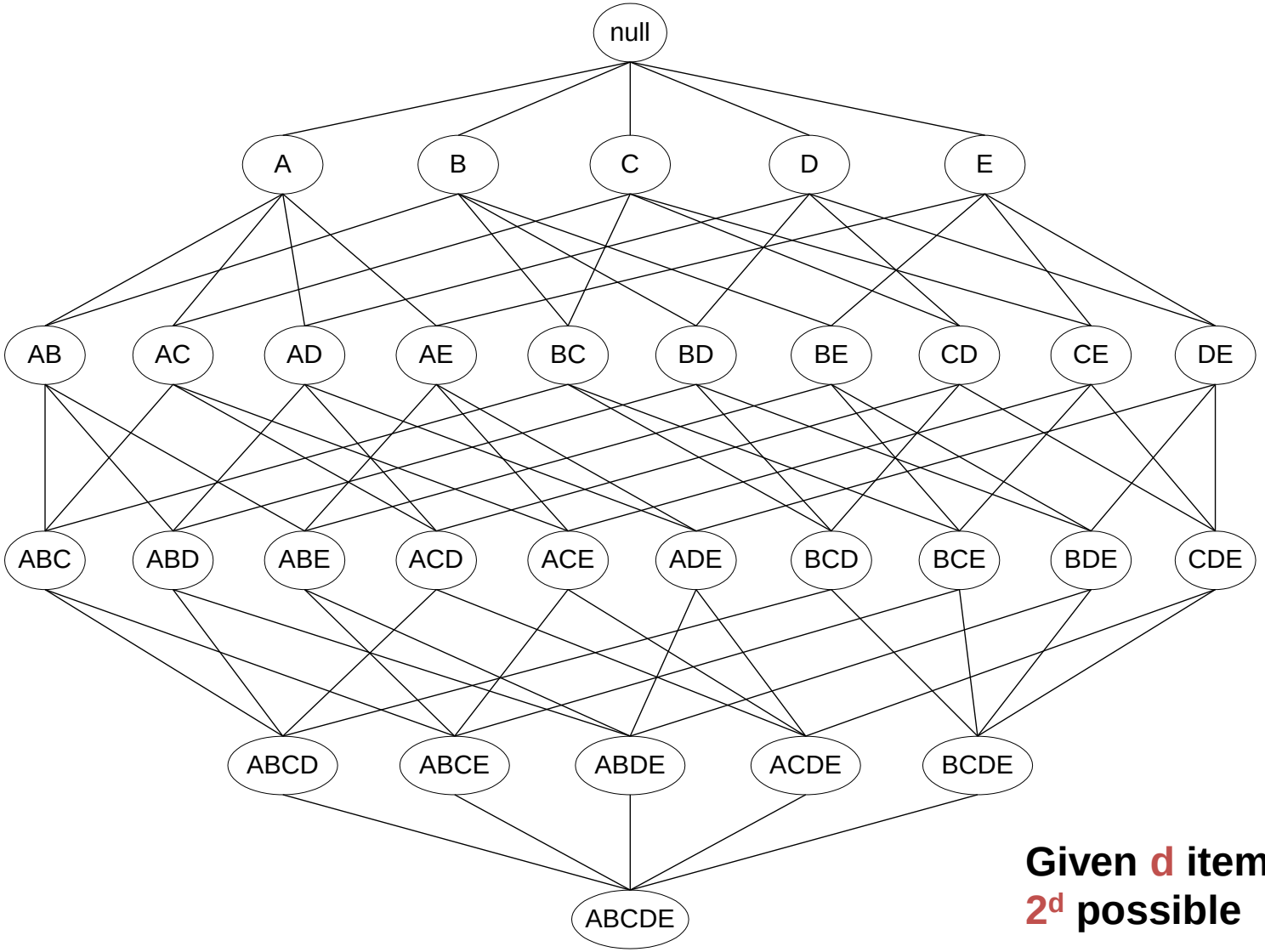
Mining Frequent Itemsets task

- Input: A set of transactions T, over a set of items I
- Output: All itemsets with items in I having
  - support ≥ *minsup* threshold

- Problem parameters:
  - N = |T|: number of transactions
  - d = |I|: number of (distinct) items
  - w: max width of a transaction
  - Number of possible itemsets? $M = 2^d$

- Scale of the problem:
  - WalMart sells 100,000 items and can store billions of baskets.
  - The Web has billions of words and many billions of pages.

The itemset lattice

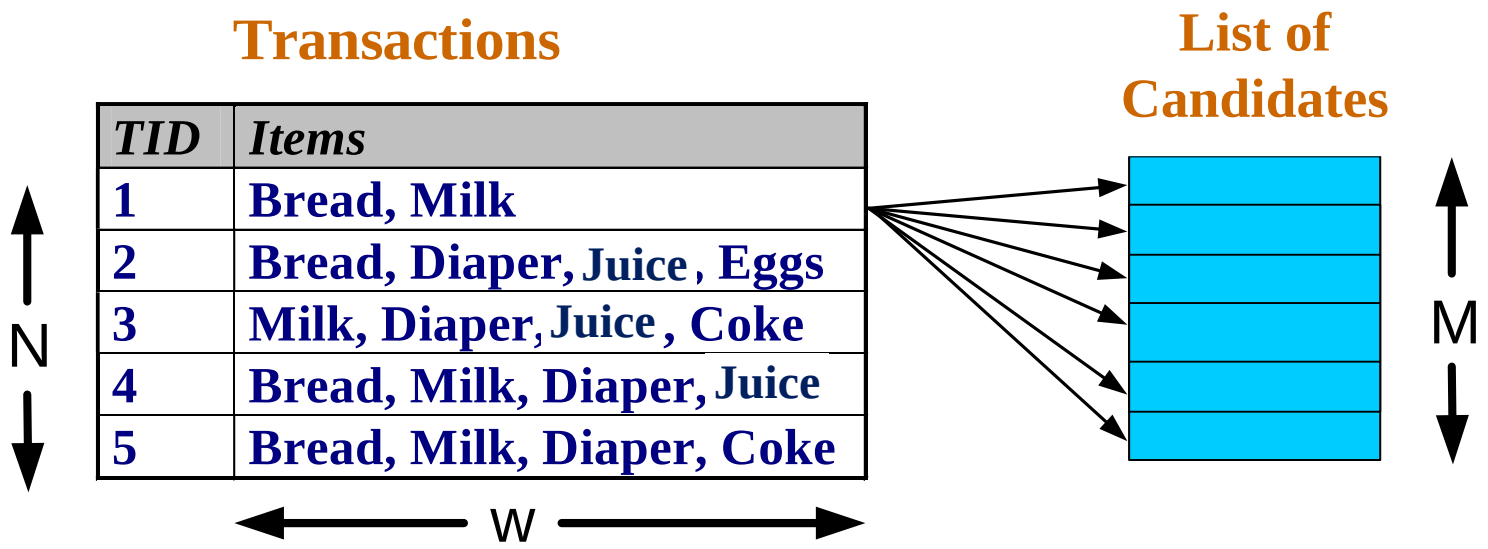


Given d items, there are 2^d possible itemsets

A Naïve Algorithm

- Brute-force approach, each itemset is a candidate :
  - Consider each itemset in the lattice, and count the support of each candidate by scanning the data
  - Time Complexity ~ O(NMw) , Space Complexity ~ O(M)

- Expensive since $M = 2^d$!!!



Computation Model

- Typically, data is kept in flat files rather than in a database system.
  - Stored on disk.
  - Stored basket-by-basket.
  - Expand baskets into pairs, triples, etc. as you read baskets.

Example file: retail

```
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29
30 31 32
33 34 35
36 37 38 39 40 41 42 43 44 45 46
38 39 47 48
38 39 48 49 50 51 52 53 54 55 56 57 58
32 41 59 60 61 62
3 39 48
63 64 65 66 67 68
32 69
48 70 71 72
39 73 74 75 76 77 78 79
36 38 39 41 48 79 80 81
82 83 84
41 85 86 87 88
39 48 89 90 91 92 93 94 95 96 97 98 99 100 101
36 38 39 48 89
39 41 102 103 104 105 106 107 108
38 39 41 109 110
39 111 112 113 114 115 116 117 118
119 120 121 122 123 124 125 126 127 128 129 130 131 132 133
48 134 135 136
39 48 137 138 139 140 141 142 143 144 145 146 147 148 149
39 150 151 152
38 39 56 153 154 155
```

Example: items are
positive integers,
and each basket corresponds to a line
in the file of space separated integers

Main-Memory Bottleneck

- For many frequent-itemset algorithms, main memory is the critical resource.
  - As we read baskets, we need to count something, e.g., occurrences of pairs.
  - The number of different things we can count is limited by main memory.
  - Swapping counts in/out is a disaster .

The Apriori Principle

- Apriori principle (Main observation):
  - If an itemset is frequent, then all of its subsets must also be frequent
  - If an itemset is not frequent, then all of its supersets cannot be frequent

$$\forall X, Y : (X \subseteq Y) \Rightarrow s(X) \geq s(Y)$$

  - The support of an itemset never exceeds the support of its subsets
  - This is known as the anti-monotone property of support

Illustration of the Apriori principle

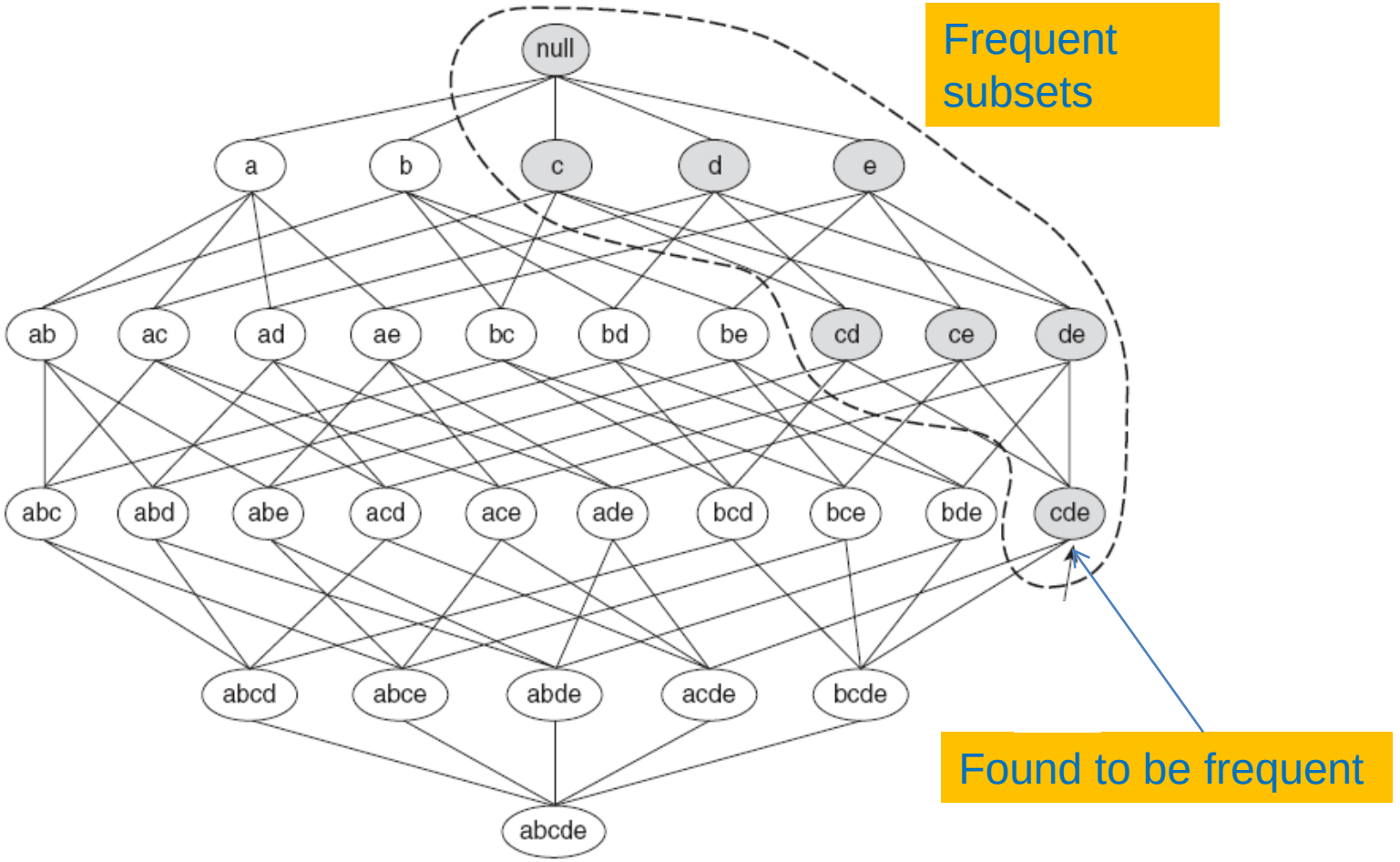
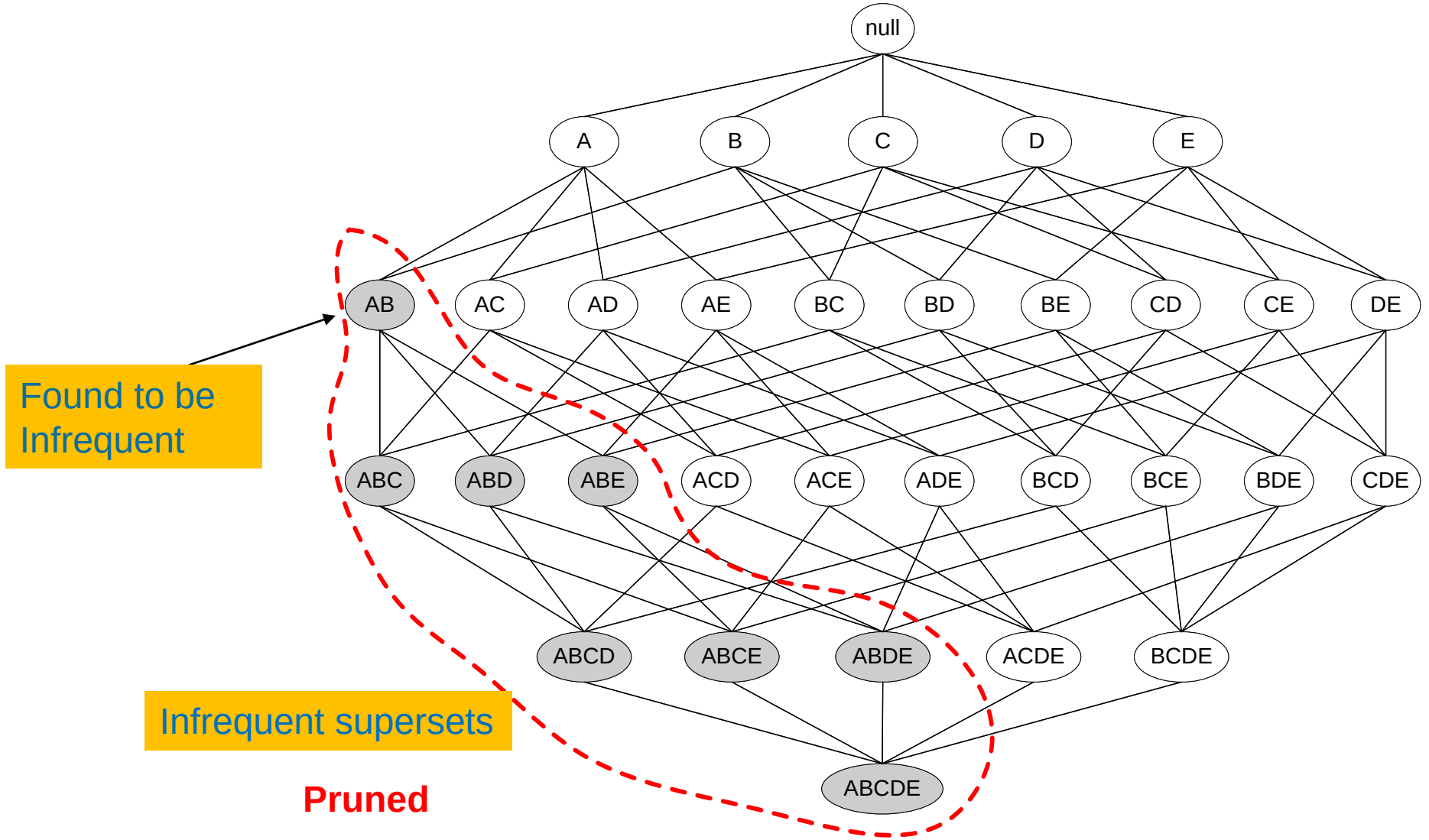


Figure 6.3. An illustration of the *Apriori* principle. If $\{c, d, e\}$ is frequent, then all subsets of this itemset are frequent.

Illustration of the Apriori principle



The Apriori algorithm

Level-wise approach

C_k = candidate itemsets of size k
L_k = frequent itemsets of size k

1. k = 1, C_1 = all items
2. While C_k not empty

Frequent itemset generation

3. Scan the database to find which itemsets in C_k are frequent and put them into L_k

Candidate generation

4. Use L_k to generate a collection of candidate itemsets C_{k+1} of size k+1

5. k = k+1

R. Agrawal, R. Srikant: "Fast Algorithms for Mining Association Rules",
Proc. of the 20th Int'l Conference on Very Large Databases, 1994.

Illustration of the Apriori principle

TID	Items
1	Bread, Milk
2	Bread, Diaper,Juice, Eggs
3	Milk, Diape,Juice,, Coke
4	Bread, Milk, Diaper ,Juice
5	Bread, Milk, Diaper, Coke

minsup = 3

Items (1-itemsets)

Item	Count
Bread	4
Coke	2
Milk	4
Juice	3
Diaper	4
Eggs	1

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Itemset	Count
{Bread,Milk}	3
{Bread, Juice}	2
{Bread,Diaper}	3
{Milk, Juice}	2
{Milk,Diaper}	3
{Juice,Diaper}	3

Triplets (3-itemsets)

Itemset	Count
{Bread,Milk,Diaper}	2

Only this triplet has all subsets to be frequent
But it is below the minsup threshold

Candidate Generation

- Basic principle (Apriori):
  - An itemset of size k+1 is candidate to be frequent only if all of its subsets of size k are known to be frequent

- Main idea:
  - Construct a candidate of size k+1 by combining frequent itemsets of size k
    - If k = 1, take the all pairs of frequent items
    - If k > 1, join pairs of itemsets that differ by just one item
    - For each generated candidate itemset ensure that all subsets of size k are frequent.

Generate Candidates C_{k+1}

- Assumption: The items in an itemset are ordered
  - E.g., if integers ordered in increasing order, if strings ordered in lexicographic order
    - The order ensures that if item y > x appears before x, then x is not in the itemset
- The items in L_k are also listed in an order

Create a candidate itemset of size k+1, by joining two itemsets of size k, that share the first k-1 items

Item 1	Item 2	Item 3
1	2	3
1	2	5
1	4	5

Item 1	Item 2	Item 3
1	2	3
1	2	5
1	4	5



1	2	3	5

Generating Candidates C_{k+1} in SQL

- ***self-join*** L_k

insert into C_{k+1}

select $p.item_1, p.item_2, \ldots, p.item_k, q.item_k$

from $L_k\ p, L_k\ q$

where $p.item_1=q.item_1, \ldots, p.item_{k-1}=q.item_{k-1}, p.item_k < q.item_k$

Example I

- L_3={abc, abd, acd, ace, bcd}

item1	item2	item3
a	b	c
a	b	d
a	c	d
a	c	e
b	c	d

p

item1	item2	item3
a	b	c
a	b	d
a	c	d
a	c	e
b	c	d

q

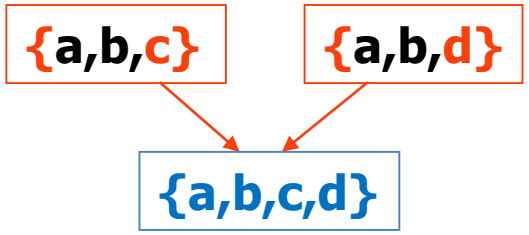
$p.item_1=q.item_1, p.item_2=q.item_2, p.item_3 < q.item_3$

Example I

- L_3={abc, abd, acd, ace, bcd}
- ***Self-joining***: L_3*L_3
  - abcd from abc and abd
  - acde from acd and ace

item1	item2	item3
a	b	c
a	b	d
a	c	d
a	c	e
b	c	d

item1	item2	item3
a	b	c
a	b	d
a	c	d
a	c	e
b	c	d



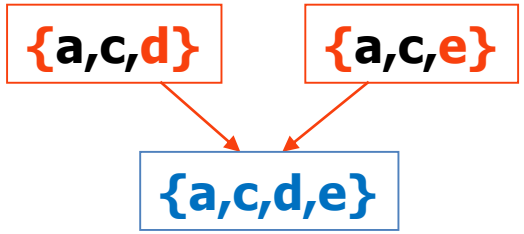
$p.item_1=q.item_1, p.item_2=q.item_2, p.item_3< q.item_3$

Example I

- L_3={abc, abd, acd, ace, bcd}
- ***Self-joining***: L_3*L_3
  - abcd from abc and abd
  - acde from acd and ace

item1	item2	item3
a	b	c
a	b	d
a	c	d
a	c	e
b	c	d

item1	item2	item3
a	b	c
a	b	d
a	c	d
a	c	e
b	c	d

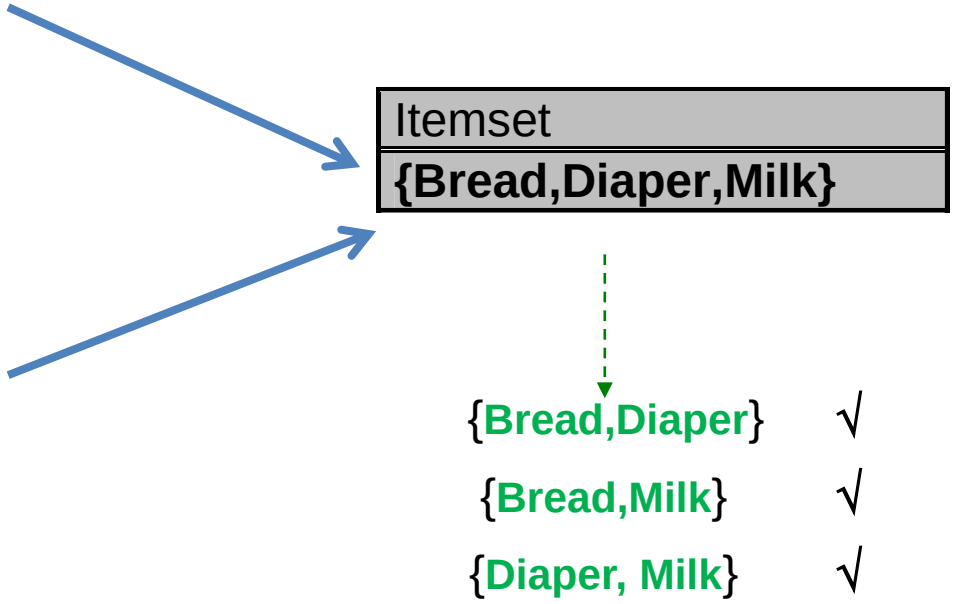


$p.item_1=q.item_1, p.item_2=q.item_2, p.item_3< q.item_3$

Example II

Itemset	Count
{Juice,Diaper}	3
{Bread,Diaper}	3
{Bread,Milk}	3
{Diaper, Milk}	3

Itemset	Count
{Juice,Diaper}	3
{Bread,Diaper}	3
{Bread,Milk}	3
{Diaper, Milk}	3



Generate Candidates C_{k+1}

- Are we done? Are all the candidates valid?

Item 1	Item 2	Item 3
1	2	3
1	2	5
1	4	5



1	2	3	5

Is this a valid candidate?

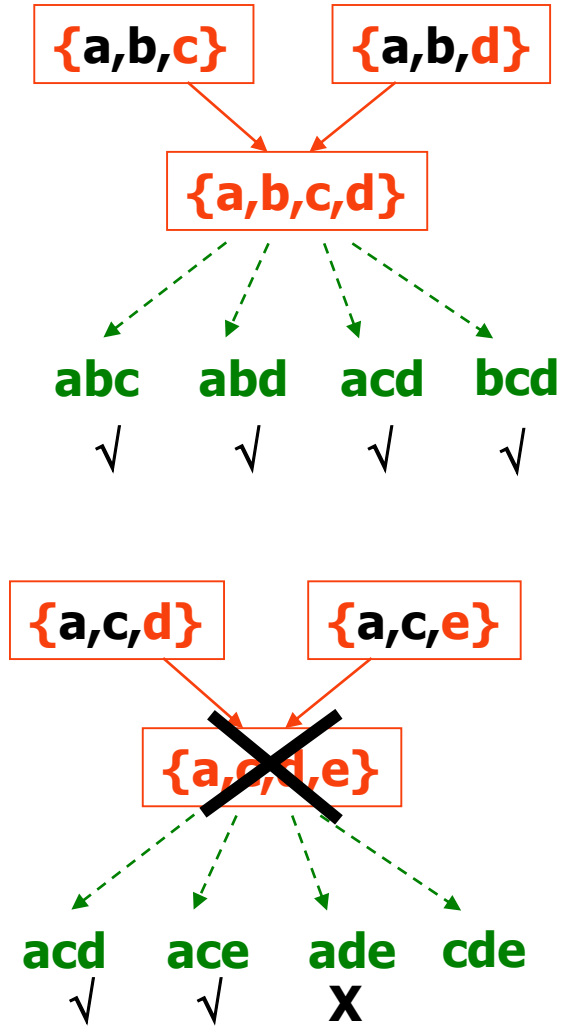
No. Subsets (1,3,5) and (2,3,5) should also be frequent

- Pruning step:
  - For each candidate (k+1)-itemset create all subset k-itemsets
  - Remove a candidate if it contains a subset k-itemset that is not frequent

Apriori principle

Example I

- L_3={*abc, abd, acd, ace, bcd*}
- ***Self-joining***: L_3*L_3
  - ***abcd*** from ***abc*** and ***abd***
  - ***acde*** from ***acd*** and ***ace***
- ***Pruning:***
  - ***abcd*** is kept since all subset itemsets are in L_3
  - ***acde*** is removed because ***ade*** is not in L_3
- C_4={*abcd*}



Generate Candidates C_{k+1}

- We have all frequent k-itemsets L_k
- **Step 1: self-join** L_k
  - Create set C_{k+1} by joining frequent k-itemsets that share the first k-1 items
- **Step 2: prune**
  - Remove from C_{k+1} the itemsets that contain a subset k-itemset that is not frequent

Is Apriori Fast Enough? — Performance Bottlenecks

- The core of the Apriori algorithm:
  - Use frequent (k – 1)-itemsets to generate candidate frequent k-itemsets
  - Use database scan and pattern matching to collect counts for the candidate itemsets
- The bottleneck of *Apriori*: candidate generation
  - Huge candidate sets:
    - 10^4 frequent 1-itemset will generate 10^7 candidate 2-itemsets
    - To discover a frequent pattern of size 100, e.g., $\{a_1, a_2, \ldots, a_{100}\}$, one needs to generate $2^{100} \approx 10^{30}$ candidates.
  - Multiple scans of database:
    - Needs (n +1) scans, n is the length of the longest pattern

Improving the Efficiency of Apriori

- “How can we further improve the efficiency of Apriori-based mining?”

Improving Apriori: general ideas

- Reduce passes of transaction database scans
- Shrink number of candidates
- Facilitate support counting of candidates

Improving Apriori's Efficiency

- Hash-based itemset counting: A *k*-itemset whose corresponding hashing bucket count is below the threshold cannot be frequent
- Transaction reduction: A transaction that does not contain any frequent k-itemset is useless in subsequent scans
- Partitioning: Any itemset that is potentially frequent in DB must be frequent in at least one of the partitions of DB
- Sampling: mining on a subset of given data, need a lower support threshold + a method to determine the completeness
- Dynamic itemset counting: add new candidate itemsets immediately (unlike Apriori) when all of their subsets are estimated to be frequent

Factors Affecting Complexity

- Choice of minimum support threshold
  - lowering support threshold results in more frequent itemsets
  - this may increase number of candidates and max length of frequent itemsets
- Dimensionality (number of items) of the data set
  - more space is needed to store support count of each item
  - if number of frequent items also increases, both computation and I/O costs may also increase
- Size of database
  - since Apriori makes multiple passes, run time of algorithm may increase with number of transactions
- Average transaction width
  - transaction width increases with denser data sets
  - This may increase max length of frequent itemsets and traversals of hash tree (number of subsets in a transaction increases with its width)

ASSOCIATION RULES

Association Rule Mining

- Given a set of transactions, find rules that will predict the occurrence of an item based on the occurrences of other items in the transaction

Market-Basket transactions

TID	*Items*
1	Bread, Milk
2	Bread, Diaper, Juice , Eggs
3	Milk, Diaper, Juice , Coke
4	Bread, Milk, Diaper, Juice
5	Bread, Milk, Diaper, Coke

Example of Association Rules

{Diaper} → {Juice},
{Milk, Bread} → {Eggs,Coke},
{Juice, Bread} → {Milk},

Implication means co-occurrence, not causality!

Definition: Association Rule

- **Association Rule**
  - An implication expression of the form $X \rightarrow Y$, where X and Y are itemsets
  - Example:
    {Milk, Diaper} $\rightarrow$ {Juice}
- Rule Evaluation Metrics
  - Support (s)
    - Fraction of transactions that contain both X and Y
    - the probability P(X,Y) that X and Y occur together
  - Confidence (c)
    - Measures how often items in Y appear in transactions that contain X
    - the conditional probability P(Y|X) that Y occurs given that X has occurred.

TID	*Items*
1	Bread, Milk
2	Bread, Diaper, Juice, Eggs
3	Milk, Diaper, Juice, Coke
4	Bread, Milk, Diaper, Juice
5	Bread, Milk, Diaper, Coke

Example:

$$\{\text{Milk}, \text{Diaper}\} \Rightarrow \text{Juice}$$

$$s = \frac{\sigma(\text{Milk}, \text{Diaper}, \text{Juice})}{|\text{T}|} = \frac{2}{5} = 0.4$$

$$c = \frac{\sigma(\text{Milk}, \text{Diaper}, \text{Juice})}{\sigma(\text{Milk}, \text{Diaper})} = \frac{2}{3} = 0.67$$

P(Y|X)=p(X,Y)/p(X)

Interestingness Measures: Support and Confidence

Example: {computer, printer}

computer ⟹ printer [support =10%, confidence= 60%]

Support =10%

10% of all the transactions under analysis show that computer and printers are purchase together

confidence= 60%

A confidence of 60% means that 60% of the customers who purchased a computer also bought the printer

Association Rule Mining Task

- Input: A set of transactions T, over a set of items I
- Output: All rules with items in I having
  - support ≥ *minsup* threshold
  - confidence ≥ *minconf* threshold

Mining Association Rules

- Two-step approach:
  1. Frequent Itemset Generation
     - Generate all itemsets whose support ≥ minsup
  2. Rule Generation
     - Generate high confidence rules from each frequent itemset, where each rule is a partitioning of a frequent itemset into Left-Hand-Side (LHS) and Right-Hand-Side (RHS)

Frequent itemset: {A,B,C,D}
Rule: AB→CD

Rule Generation

- We have all frequent itemsets, how do we get the rules?
  - For every frequent itemset S, we find rules of the form $L \rightarrow S - L$, where $L \subset S$, that satisfy the minimum confidence requirement
  - Example: $L = \{A,B,C,D\}$
  - Candidate rules:

$$A \rightarrow BCD,\quad B \rightarrow ACD,\quad C \rightarrow ABD,\quad D \rightarrow ABC$$
$$AB \rightarrow CD,\quad AC \rightarrow BD,\quad AD \rightarrow BC,\quad BD \rightarrow AC,\quad CD \rightarrow AB,$$
$$ABC \rightarrow D,\quad BCD \rightarrow A,\quad BC \rightarrow AD,$$

- If $|L| = k$, then there are $2^k - 2$ candidate association rules (ignoring $L \rightarrow \emptyset$ and $\emptyset \rightarrow L$)

Rule Generation

- How to efficiently generate rules from frequent itemsets?
  - In general, confidence does not have an anti-monotone property

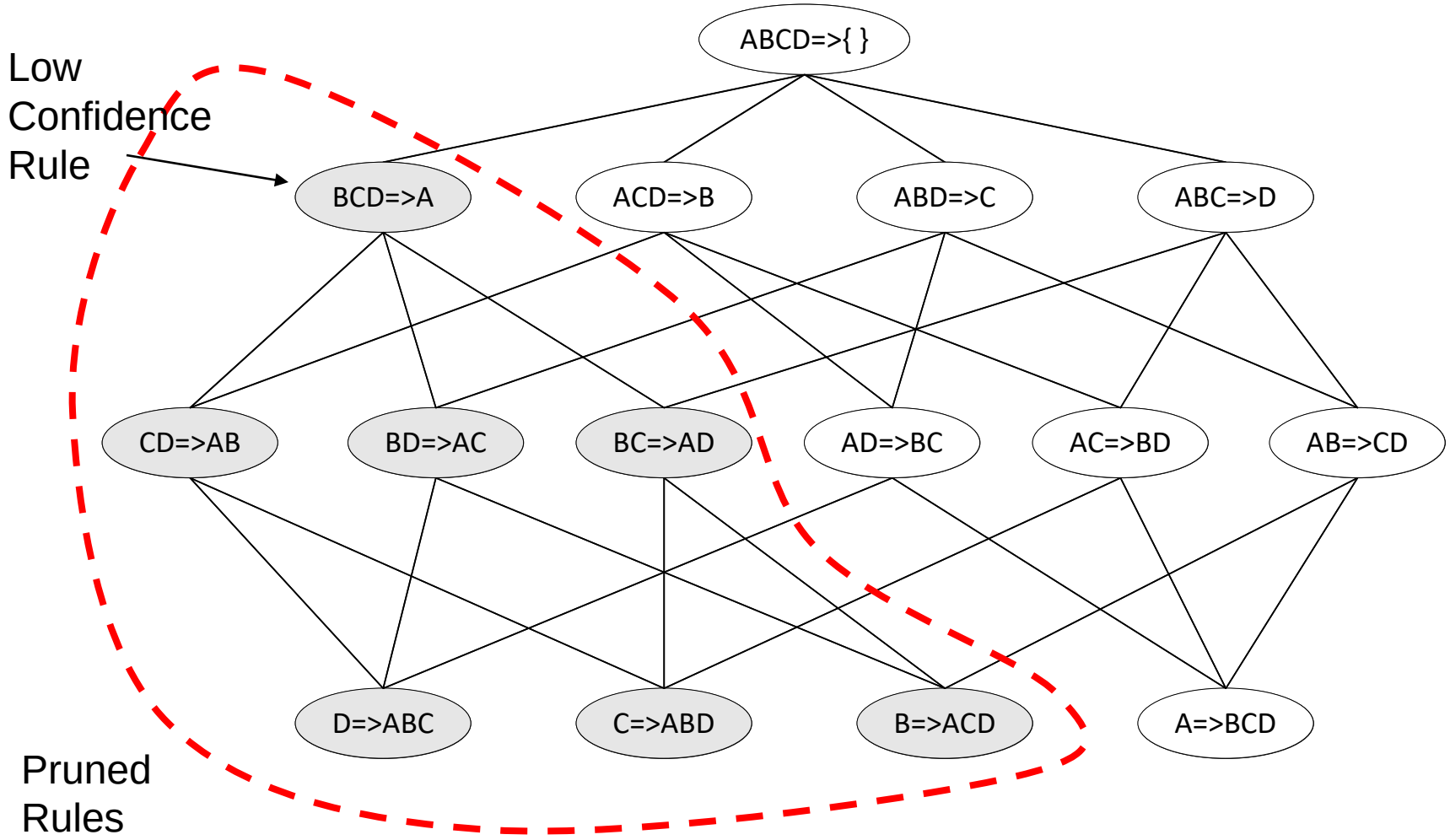
    c(ABC →D) can be larger or smaller than c(AB →D)

  - But confidence of rules generated from the same itemset has an anti-monotone property
  - e.g., L = {A,B,C,D}:

$$c(ABC \rightarrow D) \geq c(AB \rightarrow CD) \geq c(A \rightarrow BCD)$$

    - Confidence is anti-monotone w.r.t. number of items on the RHS of the rule

Rule Generation for Apriori Algorithm



Lattice of rules created by the RHS

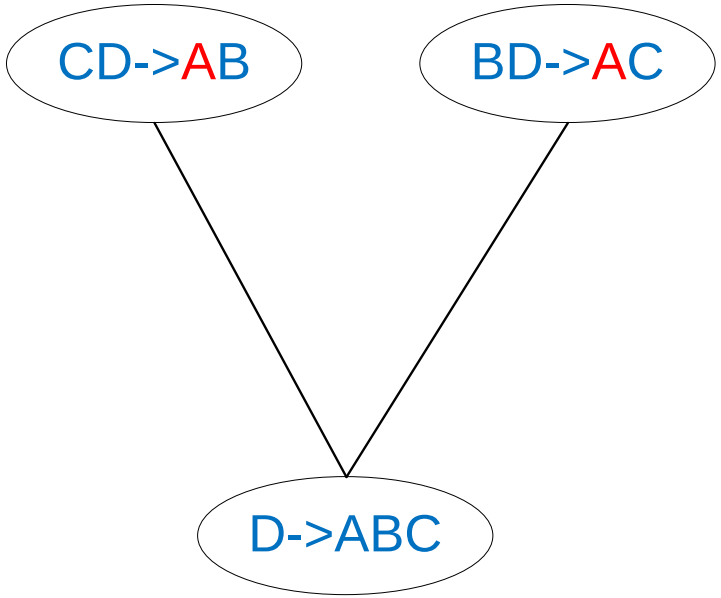
Rule Generation for APriori Algorithm

- Candidate rule is generated by merging two rules that share the same prefix
  in the RHS

- join(CD→AB,BD→AC)
  would produce the candidate
  rule D → ABC

- Prune rule D → ABC if its
  subset AD→BC does not have
  high confidence

- Essentially we are doing APriori on the RHS




THANK
YOU


Any
Question?